BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)

INTERNSHIP PROJECT REPORT

On

SPAM MAIL PREDICTION USING MACHINE LEARNING

Submitted in partial fulfilment of the requirements of the

Artificial Intelligence & Machine Learning Internship

Submitted by:

SHASHANK DESHMUKH

Under the guidance of

Mr. Md. Ameen

Naviotech Solutions Pvt Ltd

Internship Organisation

NAVIOTECH SOLUTIONS PVT LTD

Internship Duration: 13 Sept 2026 to 10 Oct 2026

Academic Year 2026 - 2027

CERTIFICATE

This is to certify that **Mr. Shashank Deshmukh**, a student of Bharati Vidyapeeth (Deemed to be University), has successfully completed the internship in **Artificial Intelligence & Machine Learning** at **Naviotech Solutions Pvt Ltd** during the period from 13 Sept 2026 to 10 Oct 2026.

During the internship, the candidate completed the project: Spam Mail Prediction using Machine Learning. The project work reported in this document was carried out sincerely under our guidance. It is original, and the candidate showed a good understanding of the data pre-processing, model building, evaluation and documentation involved.

We wish him success in all his future endeavours.

_____ _____

Mr. Md. Ameen
Internship Guide
Naviotech Solutions Pvt Ltd

Academic Head
Naviotech Solutions Pvt Ltd

Date: 29/10/2026

Place: Bhilai

(Institute seal)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during my internship and in completing this project.

First, I thank my internship guide, Mr. Md. Ameen, for his valuable guidance, patient explanations and constant encouragement. His suggestions on how to structure a machine learning project, from data understanding to evaluation, helped me greatly in completing this work.

I am grateful to Naviotech Solutions Pvt Ltd for giving me the opportunity to work on practical Artificial Intelligence and Machine Learning projects. The hands-on exposure I received, first with Customer Churn Prediction and then with this Spam Mail Prediction project, gave me real confidence in using Python and its data science libraries.

I also thank Bharati Vidyapeeth (Deemed to be University) and its faculty for providing the academic foundation on which this work is built. I thank the creators of the SMS Spam Collection dataset and the open-source community behind Python, scikit-learn, pandas, matplotlib and Jupyter, whose tools made this project possible.

Finally, I thank my family and friends for their support and motivation throughout the internship.

Shashank Deshmukh

ABSTRACT

Unwanted and fraudulent messages, commonly called spam, waste users' time and expose them to phishing, financial scams and malware. Filtering such messages automatically is one of the most widely deployed applications of machine learning. This project builds a spam mail prediction system that classifies a text message as either spam or ham (a legitimate message).

The system uses the SMS Spam Collection dataset, which contains 5,169 unique messages after duplicate removal (4,516 ham and 653 spam). Each message is cleaned by converting it to lowercase, replacing URLs and long phone numbers with special tokens, removing punctuation and removing English stop-words. The cleaned text is converted into numerical features with TF-IDF (Term Frequency - Inverse Document Frequency) using unigrams and bigrams, producing a vocabulary of 5,476 features.

Four classifiers were trained on 80% of the data and tested on the remaining 20%: Multinomial Naive Bayes, Logistic Regression, Linear Support Vector Machine and Random Forest. They were compared using accuracy, precision, recall, F1-score and ROC-AUC. Multinomial Naive Bayes gave the best overall result, with an accuracy of 98.74%, a precision of 98.36%, a recall of 91.60% and an F1-score of 94.86% on unseen messages. Five-fold cross-validation gave a mean accuracy of 98.59%, which confirms that the result is stable and not due to a lucky split.

The final model is saved and can classify any new message in a fraction of a second. The project shows that a simple, well-prepared text classification pipeline can achieve high accuracy, remain easy to explain, and be deployed with little computing power.

Keywords: Spam detection, Text classification, NLP, TF-IDF, Naive Bayes, Machine Learning, Python, scikit-learn.

TABLE OF CONTENTS

Certificate 2
Acknowledgement 3
Abstract 4
Chapter 1: Introduction 6
Chapter 2: Literature Review 8
Chapter 3: Methodology 10
Chapter 4: System Design 14
Chapter 5: Implementation 16
Chapter 6: Results and Discussion 19
Chapter 7: Conclusion 25
Chapter 8: Future Scope 26
References 27
Appendix: Complete Source Code 28

CHAPTER 1: INTRODUCTION

1.1 Background

Electronic communication through email, SMS and messaging apps is part of everyday life. Along with genuine messages, users receive a large amount of unsolicited and often harmful content called spam. Spam messages advertise unwanted products, announce fake lottery wins, push users to premium-rate phone numbers or lead them to fraudulent websites that steal passwords and bank details. Reports from security organisations consistently show that a large share of the world's email traffic is spam, and mobile spam is growing quickly as phones become the main way people communicate.

Spam is harmful in three ways. It wastes time and attention, because users must sort through junk to find real messages. It costs money, since networks must carry and store the extra traffic and users may be charged for replying to premium-rate numbers. Most seriously, it is a security risk, because many spam messages are phishing attempts that pretend to come from banks, delivery companies or government offices.

Manually blocking spam is not practical at the scale of modern networks. Early filters used fixed rules, such as blocking messages that contained certain words. Spammers quickly learned to change spellings (for example writing 'fr33' instead of 'free'), which made rule lists outdated almost at once. Machine learning offers a better approach: instead of being told what spam looks like, the system learns the patterns from thousands of labelled examples and can adapt when it is retrained with new data.

1.2 Problem Statement

Given a text message, build a machine learning model that automatically decides whether the message is spam or ham (legitimate). The model must be accurate on messages it has never seen, it must rarely block genuine messages (a low false-positive rate), and it must run fast enough to be used in a real system.

This is a binary text classification problem. The input is free-form text of variable length, written informally with abbreviations, slang, numbers and symbols. The output is one of two labels: spam or ham.

1.3 Objectives

- To study and understand the SMS Spam Collection dataset and the imbalance between spam and ham messages.

- To clean and pre-process raw text so that a machine learning algorithm can use it.
- To convert text into numerical features using the TF-IDF technique.
- To train and compare four machine learning classifiers: Naive Bayes, Logistic Regression, Linear SVM and Random Forest.
- To evaluate the models with accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC, and to select the best one.
- To analyse the mistakes made by the model and understand why they happen.
- To save the best model and use it to predict the class of new messages.

1.4 Scope of the Project

The project covers the complete machine learning workflow for a text classification task: data collection, exploratory data analysis, text cleaning, feature extraction, model training, evaluation and prediction. It uses the English-language SMS Spam Collection dataset. The same pipeline can be applied to email subjects and bodies with a suitable dataset. The project does not include building a mobile app or integrating with a mail server; these are discussed under Future Scope.

1.5 Motivation

Spam detection is a classic and practical machine learning problem. It contains most of the ideas that appear in many Natural Language Processing (NLP) tasks, including tokenisation, stop-word removal, vectorisation and the handling of imbalanced classes. Because the output is easy to understand (a message is either spam or it is not), the project is also a good way to learn how to evaluate and explain a model. After completing a tabular project (Customer Churn Prediction), this project was chosen to gain experience with text data, which needs a different kind of pre-processing.

1.6 Organisation of the Report

Chapter 2 reviews earlier work on spam filtering. Chapter 3 explains the dataset, the pre-processing steps, the algorithms and the evaluation measures. Chapter 4 presents the system design. Chapter 5 describes the implementation with code. Chapter 6 presents and discusses the results. Chapter 7 concludes the report and Chapter 8 suggests future improvements. References and the complete source code are given at the end.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

Spam filtering has been studied for more than two decades, and the methods used have moved from hand-written rules to statistical learning and, more recently, to deep neural networks. This chapter summarises the main approaches and the research that shaped them.

2.2 Rule-Based and Blacklist Filters

The first spam filters relied on manually written rules and lists. A rule might block a message that contains a phrase such as 'free money', or that comes from a sender or IP address on a blacklist. These filters are easy to understand, but they need constant manual updates. Spammers can defeat them by changing wording, adding random characters or using new sender addresses. Rule-based systems also generate many false positives, because genuine messages sometimes contain the same words.

2.3 Statistical and Bayesian Filters

A major step forward came when researchers treated spam filtering as a probability problem. Sahami et al. (1998) proposed a Bayesian approach to junk email filtering that estimated, from the words in a message, how likely it was to be junk. Androutsopoulos et al. (2000) evaluated Naive Bayes filters on public email collections and showed that they work well in practice. Paul Graham's essay 'A Plan for Spam' (2002) made Bayesian filtering popular among everyday email systems, because it was accurate, adapted to each user, and hard for spammers to avoid.

Naive Bayes remains a strong baseline for text classification because it is fast, needs little data, handles a large vocabulary well and gives probabilities that are easy to interpret.

2.4 Support Vector Machines and Other Classical Models

Drucker, Wu and Vapnik (1999) applied Support Vector Machines (SVM) to spam categorisation and found them to be among the best performing methods, particularly for high-dimensional sparse text features. Logistic Regression and ensemble methods such as Random Forests (Breiman, 2001) have also been widely used. These models usually work with word-count or TF-IDF features. TF-IDF itself comes from the information retrieval work of Salton and Buckley (1988), who showed that weighting a term by how rare it is across documents improves search and classification.

2.5 The SMS Spam Collection

Almeida, Gomez Hidalgo and Yamakami (2011) created and published the SMS Spam Collection, a labelled set of real SMS messages collected from several public sources. It is now a standard benchmark for short-text spam filtering. SMS spam is harder than email spam in some ways: messages are short, informal, full of abbreviations, and contain fewer words from which to make a decision. The authors tested several classifiers and found that SVM and Naive Bayes based methods produced strong results.

2.6 Deep Learning Approaches

More recent work uses neural networks. Recurrent networks such as LSTM (Hochreiter and Schmidhuber, 1997) read a message word by word and can capture word order, while transformer models such as BERT (Devlin et al., 2019) learn rich representations of language from very large text collections. These models can reach slightly higher accuracy and handle disguised spam better, but they need much more computing power, more training time and are harder to explain. For a small dataset of about five thousand messages, classical models with good features are often almost as accurate and far cheaper to run.

Table 2.1 Comparison of spam filtering approaches

Approach	Idea	Strengths	Limitations
Rule / blacklist	Fixed keywords and sender lists	Simple, transparent	Easy to evade; manual updates
Naive Bayes	Word probabilities per class	Fast, works with small data	Assumes words are independent
Logistic Regression	Linear model with probabilities	Stable, interpretable weights	Cannot learn word interactions
SVM	Maximum-margin boundary	Strong on sparse text	Less probabilistic output
Random Forest	Many decision trees voting	Handles non-linear patterns	Slower on large sparse text
LSTM / BERT	Neural language models	Capture context and word order	Heavy compute, less explainable

2.7 Research Gap and Contribution of This Work

Many studies report only accuracy. On a dataset where only about 13% of messages are spam, accuracy alone can be misleading, since a model that always answers 'ham' would already be 87% accurate. This project therefore reports precision, recall, F1-score, the confusion matrix and ROC-AUC, uses stratified splitting and five-fold cross-validation,

and studies the model's mistakes in detail. It also compares four models under identical conditions so that the choice of the final model is supported by evidence.

CHAPTER 3: METHODOLOGY

3.1 Overview of the Approach

The work follows the standard machine learning workflow: (1) collect and understand the data, (2) clean the text, (3) convert the text to numbers, (4) split the data into training and testing sets, (5) train several models, (6) evaluate them on unseen data, (7) choose and save the best model, and (8) use it to predict new messages. Each step is described below.

3.2 Dataset Description

The SMS Spam Collection dataset contains 5,572 messages, each labelled 'ham' (legitimate) or 'spam'. The data was read directly from a public tab-separated file with two columns: label and message. There are no missing values. However, 403 messages were exact duplicates, so they were removed to prevent the same message from appearing in both the training and testing sets, which would give unrealistically good results. After this step, 5,169 messages remained.

Table 3.1 Summary of the dataset

Property	Value
Source	SMS Spam Collection (UCI Machine Learning Repository)
Total messages (raw)	5,572
Duplicate messages removed	403
Messages used	5,169
Ham (legitimate)	4,516 (87.4%)
Spam	653 (12.6%)
Average length of a ham message	70.9 characters
Average length of a spam message	137.7 characters
Columns	label (ham/spam), message (text)

The data is imbalanced: spam forms only about one in eight messages. This is realistic, since most messages people receive are genuine. To ensure that both the training and testing sets contain the same proportion of spam, a stratified split was used. Class weights were also applied in Logistic Regression and Linear SVM, so that mistakes on the minority spam class are penalised more.

A useful observation from the data is that spam messages are on average almost twice as long as ham messages. Spam typically includes an offer, a condition and a call to action

(for example a number to call or a code to text), while genuine messages are short and personal.

3.3 Text Pre-processing

Raw text cannot be given directly to a machine learning algorithm. It is first cleaned to reduce noise and make similar messages look similar. The following steps were applied in order:

1. **Lowercasing.** All text is converted to lowercase so that 'FREE', 'Free' and 'free' are treated as the same word.
2. **URL replacement.** Web links are replaced by the token 'urltoken'. The exact link does not matter, but the presence of a link is a strong spam signal.
3. **Long-number replacement.** Numbers with five or more digits (phone numbers, short codes, prize amounts) are replaced by 'longnumber'. Again, the presence of such a number is informative, but the exact digits are not.
4. **Punctuation removal.** Punctuation marks are removed.
5. **Stop-word removal.** Very common English words such as 'the', 'is' and 'and' are removed, because they appear in both classes and carry little meaning. The stop-word list from scikit-learn was used.

An example from the dataset shows the effect. The original message 'Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's' becomes 'free entry 2 wkly comp win fa cup final tkts 21st 2005 text fa longnumber receive entry questionstd txt ratetcs apply longnumber over18s'. Words were deliberately not stemmed or lemmatised, because spam often uses unusual word forms (for example 'wkly', 'tkts', 'txt') that are themselves useful signals.

3.4 Feature Extraction using TF-IDF

After cleaning, each message is still a string. To feed it to a model, it is converted to a vector of numbers using TF-IDF (Term Frequency - Inverse Document Frequency). The idea is to give a high weight to words that are frequent in a message but rare in the rest of the collection, because such words describe the message well.

Term Frequency (TF) measures how often a word appears in a message. **Inverse Document Frequency (IDF)** measures how rare the word is across all messages, computed as IDF(t) = log(N / df(t)), where N is the number of messages and df(t) is the number of

messages containing the term t. The TF-IDF weight is the product TF x IDF. A word like 'prize' is rare in normal conversation, so it gets a high weight, while a word that appears in nearly every message gets a low weight.

The vectoriser was configured with unigrams and bigrams (single words and pairs of consecutive words), a minimum document frequency of 2 (words that occur in only one message are ignored) and sublinear term frequency scaling (1 + log TF), which prevents a word repeated many times from dominating. The result is a sparse matrix with 5,476 features. Bigrams help capture phrases such as 'call now' or 'free entry' that carry more meaning together than separately.

3.5 Train-Test Split

The data was divided into a training set (80%, 4,135 messages) and a testing set (20%, 1,034 messages) using stratified sampling with a fixed random seed of 42 so that the results can be reproduced. The model learns only from the training set. The testing set is kept aside and used once, at the end, to measure how well the model works on messages it has never seen. The TF-IDF vectoriser is fitted only on the training data inside a scikit-learn Pipeline, which prevents information from the test set leaking into training.

3.6 Machine Learning Algorithms Used

3.6.1 Multinomial Naive Bayes

Naive Bayes applies Bayes' theorem, P(class | message) = P(message | class) x P(class) / P(message), together with the 'naive' assumption that words are independent of each other given the class. For text, the Multinomial version models how many times each word occurs in each class. To classify a message, the model multiplies the probabilities of its words under each class (in practice it adds logarithms) and chooses the class with the higher score. A smoothing parameter alpha (set to 0.1) prevents a word never seen in one class from making the probability zero. Although the independence assumption is not true for real language, the method works very well in practice and is extremely fast.

3.6.2 Logistic Regression

Logistic Regression is a linear model that computes a weighted sum of the features and passes it through the sigmoid function to produce a probability between 0 and 1. The weights are learned by minimising the log loss. A larger weight on a word means that the word pushes the message towards spam. The regularisation parameter C was set to 10 and class weights were balanced so that the minority class is not ignored.

3.6.3 Linear Support Vector Machine

A Linear SVM finds the straight boundary (hyperplane) that separates the two classes with the largest possible margin, which usually generalises well. Text data with thousands of sparse features is often linearly separable, so linear SVMs are a natural fit. LinearSVC from scikit-learn was used with C = 1 and balanced class weights.

3.6.4 Random Forest

A Random Forest builds many decision trees, each on a random sample of the data and a random subset of features, and combines their votes. Averaging many different trees reduces overfitting. A forest of 200 trees was used. Random Forests can model non-linear relationships but are usually slower and less efficient than linear models on very high-dimensional sparse text.

3.7 Evaluation Metrics

Because the classes are imbalanced, several metrics were used. Let TP be spam correctly detected, TN ham correctly passed, FP ham wrongly marked as spam and FN spam that was missed.

Table 3.2 Evaluation metrics

Metric	Formula	Meaning in this project
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Share of all messages classified correctly
Precision	TP / (TP + FP)	Of the messages marked spam, how many really are spam
Recall	TP / (TP + FN)	Of all real spam, how much was caught
F1-score	2 x Precision x Recall / (Precision + Recall)	Balance between precision and recall
ROC-AUC	Area under the ROC curve	Ability to rank spam above ham at any threshold

In a spam filter, precision is especially important, because a false positive means a genuine message (perhaps a bank alert or a job offer) is hidden from the user. Recall matters too, because a missed spam message is an annoyance and a possible scam. The F1-score balances the two and was used to select the best model.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The system is organised as a pipeline. Data flows from left to right through the stages of cleaning, feature extraction, training and evaluation. The best model is then saved to a file and reused for predicting new messages. The architecture is shown in Figure 4.1.

Figure 4.1 System Architecture of the Spam Mail Prediction System

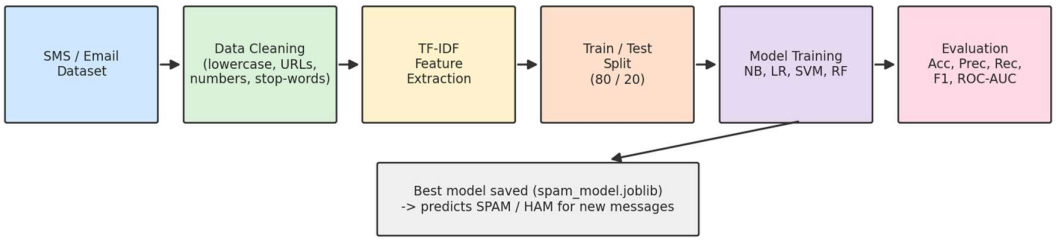


Figure 4.1 System architecture of the spam mail prediction system

4.2 Module Description

Table 4.1 Modules of the system

Module	Description	Output
Data Loading	Reads the tab-separated file with pandas, checks shape, missing values and duplicates	Clean DataFrame
Exploratory Analysis	Class distribution, message length and most frequent words per class	Charts and insights
Text Cleaning	clean_text() function: lowercase, URL/number tokens, punctuation and stop-word removal	Cleaned text column
Feature Extraction	TfidfVectorizer with unigrams and bigrams	Sparse TF-IDF matrix
Model Training	Four classifiers inside scikit-learn Pipelines	Trained models
Evaluation	Accuracy, precision, recall, F1, confusion matrix, ROC-AUC, cross-validation	Metrics and plots
Prediction	Loads the saved model and classifies a new message	SPAM or HAM label

4.3 Prediction Flow

When a new message arrives, it passes through exactly the same cleaning function and the same fitted TF-IDF vectoriser that were used during training. It is then given to the classifier, which returns the label. Using the same steps is essential: if the new message were processed differently from the training data, the features would not match and the prediction would be unreliable. Wrapping the vectoriser and the classifier into one Pipeline object guarantees that they are always used together.

Figure 4.2 Prediction Flow for a New Message



Figure 4.2 Prediction flow for a new message

4.4 Hardware and Software Requirements

Table 4.2 Hardware and software requirements

Item	Requirement
Processor	Any modern dual-core CPU (no GPU needed)
RAM	4 GB or more
Operating system	Windows / Linux / macOS
Language	Python 3.9 or higher
Libraries	pandas, numpy, scikit-learn, matplotlib, seaborn, joblib
Environment	Google Colab or Jupyter Notebook
Dataset	SMS Spam Collection (tab-separated text file)

The whole pipeline trains in a few seconds on an ordinary laptop or on the free tier of Google Colab, which shows that a useful spam filter does not need expensive hardware.

4.5 Design Decisions

- TF-IDF with bigrams was chosen over simple word counts because it down-weights common words and captures short phrases.
- Stemming was avoided because unusual spellings are themselves a spam signal.
- Pipelines were used so that the vectoriser is fitted only on training data, preventing data leakage.
- A stratified split with a fixed seed was used for fair and reproducible comparison.
- Four different families of models were compared (probabilistic, linear, margin-based and ensemble) to avoid choosing a model without evidence.

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

The project was implemented in Python using Google Colab and Jupyter Notebook. The libraries used are pandas and numpy for data handling, scikit-learn for feature extraction, modelling and evaluation, matplotlib and seaborn for visualisation, and joblib for saving the trained model. The code is also supplied as a single Python script (spam_detection.py) and as a notebook (Spam_Mail_Prediction.ipynb).

5.2 Step 1: Loading and Inspecting the Data

The dataset is read with pandas using a tab separator, because the file is tab-separated and has no header row. The shape, the count of each class, missing values and duplicates are checked. Duplicates are dropped and the text labels are converted to numbers (ham = 0, spam = 1), which is the format scikit-learn expects.

```
import re, string
import numpy as np, pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB

# 1. Load the data
URL = "https://raw.githubusercontent.com/justmarkham/pycon-2016-
tutorial/master/data/sms.tsv"
df = pd.read_csv(URL, sep="\t", header=None, names=["label", "message"])
df = df.drop_duplicates().reset_index(drop=True)
df["target"] = df["label"].map({"ham": 0, "spam": 1})
```

5.3 Step 2: Exploratory Data Analysis

Charts were created to understand the data before modelling. A bar chart and a pie chart show the class balance. A histogram compares message lengths of the two classes. Bar charts of the most frequent words in each class show which words are typical for spam and for ham. These results are shown in Chapter 6.

5.4 Step 3: Text Cleaning

The clean_text function implements the pre-processing steps described in Section 3.3. It is applied to every message with pandas apply(), and the cleaned text is stored in a new column so that the original message is still available for inspection.

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\.\S+", " urltoken ", text)
    text = re.sub(r"\d{5,}", " longnumber ", text)
    text = text.translate(str.maketrans("", "", string.punctuation))
    return " ".join(w for w in text.split() if w not in ENGLISH_STOP_WORDS)
```

```
df["clean"] = df["message"].apply(clean_text)
```

5.5 Step 4: Splitting, Feature Extraction and Training

The data is split with a stratified 80/20 split. Each model is built as a Pipeline that first applies the TF-IDF vectoriser and then the classifier. Calling fit() on the pipeline fits the vectoriser and the classifier on the training data only, and calling predict() on the pipeline applies the already fitted vectoriser to the test text. The same pattern was repeated for all four classifiers.

```
X_train, X_test, y_train, y_test = train_test_split(
    df["clean"], df["target"], test_size=0.2,
    stratify=df["target"], random_state=42)

def make_pipe(clf):
    return Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2,
sublinear_tf=True)),
        ("clf", clf)])

model = make_pipe(MultinomialNB(alpha=0.1))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

5.6 Step 5: Evaluation

For every model the script calculates accuracy, precision, recall, F1-score and ROC-AUC on the test set, collects them in a results table, and selects the best model by F1-score. For the best model, it also prints the classification report, draws the confusion matrix and plots the ROC curves of all models. Five-fold stratified cross-validation was run separately to check the stability of the result.

5.7 Step 6: Saving the Model and Predicting New Messages

The best pipeline is saved with joblib as spam_model.joblib. Because the pipeline contains both the vectoriser and the classifier, loading this single file is enough to make predictions later. A helper function cleans a new message, passes it to the model and returns a readable label.

```
def predict_message(msg):
    label = model.predict([clean_text(msg)])[0]
    return "SPAM" if label == 1 else "HAM (not spam)"

print(predict_message("URGENT! Call 09061701461 to claim your cash prize"))
```

5.8 Challenges Faced and Solutions

Table 5.1 Challenges and solutions

Challenge	Solution
Class imbalance (about 13% spam)	Stratified split, class weights, and evaluation with precision, recall and F1 instead of accuracy alone
Duplicate messages could leak into the test set	Removed 403 exact duplicates before splitting
Informal text with abbreviations and numbers	Replaced URLs and long numbers with tokens; avoided stemming; used bigrams
Risk of data leakage through the vectoriser	Used scikit-learn Pipelines so TF-IDF is fitted on training data only
Choosing the best model fairly	Trained all models on the same split and compared them with the same metrics, plus cross-validation

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Exploratory Data Analysis Results

Figure 6.1 shows the class distribution. Of 5,169 unique messages, 4,516 (87.4%) are ham and 653 (12.6%) are spam, so the dataset is clearly imbalanced.

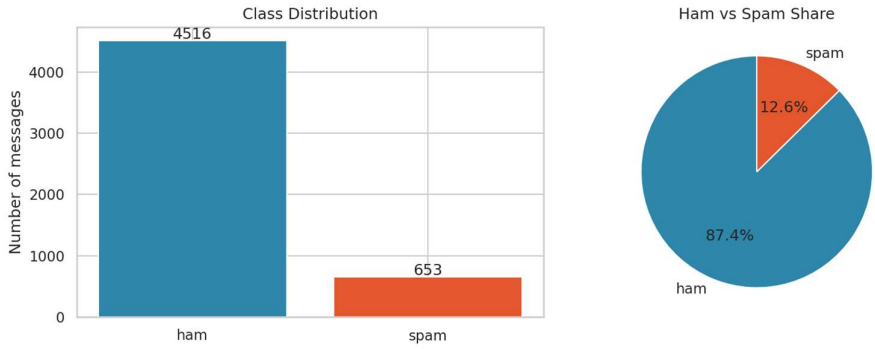


Figure 6.1 Class distribution of the dataset

Figure 6.2 compares message lengths. Ham messages average 70.9 characters, while spam messages average 137.7. Spam therefore tends to be about twice as long, and message length by itself is already a useful hint.

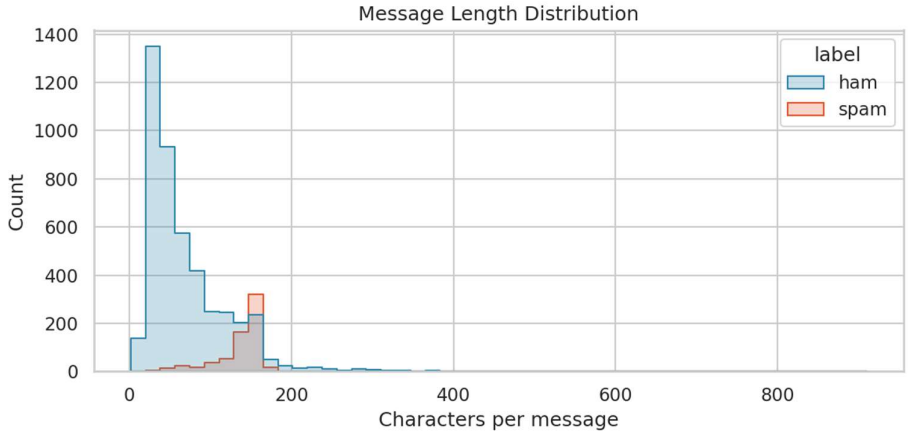


Figure 6.2 Distribution of message length for ham and spam

Figure 6.3 lists the most frequent words in each class after cleaning. Ham is dominated by everyday conversational words, while spam contains words that urge the reader to act, such as 'free', 'txt', 'text', 'claim', 'mobile' and 'reply', together with the long-number token.

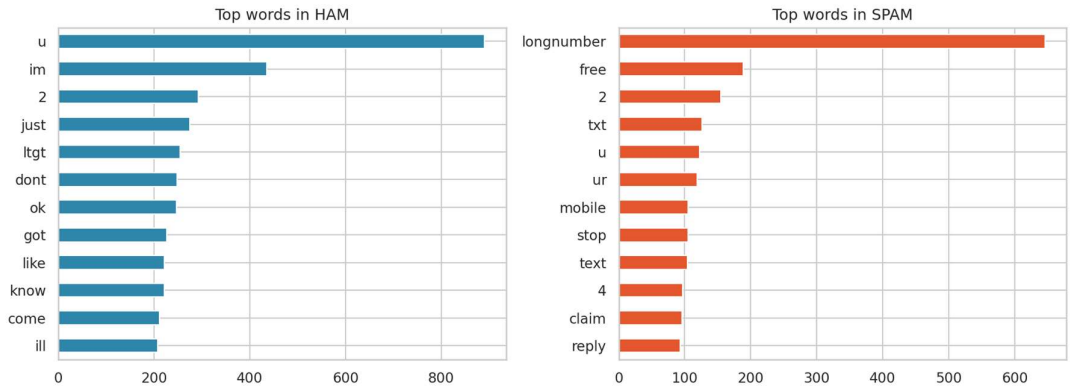


Figure 6.3 Most frequent words in ham and spam messages

6.2 Model Comparison

All four models were trained on the same 4,135 training messages and tested on the same 1,034 unseen test messages. Table 6.1 shows the results, sorted by F1-score.

Table 6.1 Performance of all models on the test set

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Naive Bayes	98.74%	98.36%	91.60%	94.86%	0.9905
Random Forest	98.55%	100.00%	88.55%	93.93%	0.9931
Logistic Regression	98.36%	95.24%	91.60%	93.39%	0.9948
Linear SVM	98.36%	96.72%	90.08%	93.28%	0.9947

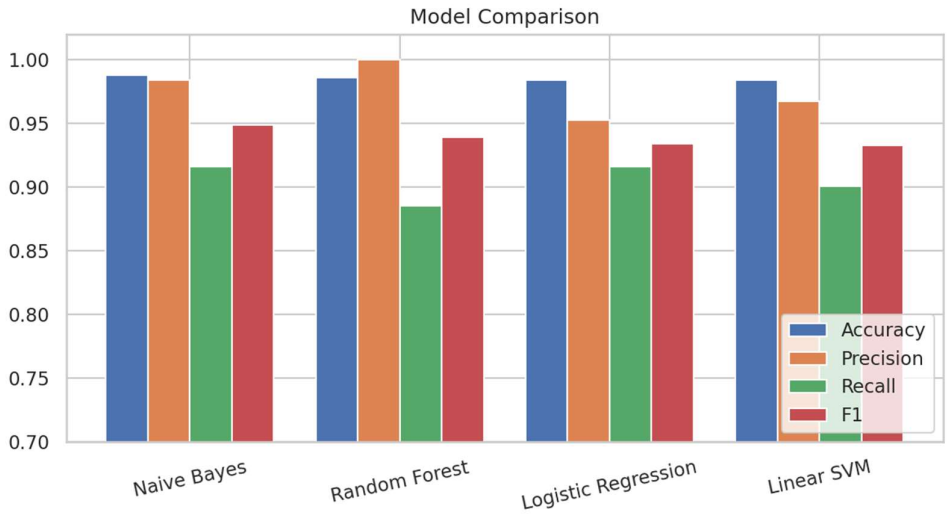


Figure 6.4 Comparison of the four models

All four models achieve an accuracy between 98.36% and 98.74%, which shows that the TF-IDF features separate the two classes very well. The differences are therefore small, and the choice depends on the balance between precision and recall. Random Forest has

perfect precision (no genuine message was blocked) but the lowest recall, so it misses more spam. Logistic Regression has the highest ROC-AUC, meaning it ranks messages best across all thresholds. Multinomial Naive Bayes has the best F1-score and the best accuracy, with both high precision and the joint highest recall, and it is also the fastest to train and to run. It was therefore selected as the final model.

6.3 Detailed Results of the Best Model (Naive Bayes)

6.3.1 Confusion Matrix

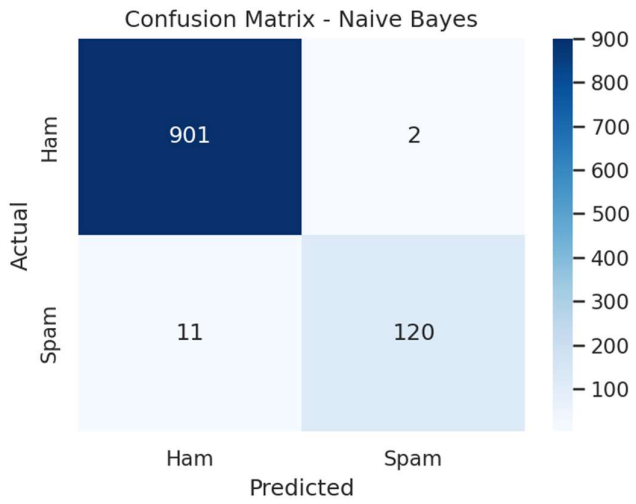


Figure 6.5 Confusion matrix of the Naive Bayes model

Table 6.2 Confusion matrix values

	Predicted Ham	**Predicted Spam**
Actual Ham	901 (true negatives)	2 (false positives)
Actual Spam	11 (false negatives)	120 (true positives)

Out of 903 genuine messages in the test set, 901 were correctly passed and only 2 were wrongly marked as spam. Out of 131 spam messages, 120 were caught and 11 were missed. The model therefore rarely blocks genuine messages, which is the most important property of a spam filter.

6.3.2 Classification Report

```
precision    recall  f1-score   support

         Ham     0.9879    0.9978    0.9928       903
        Spam     0.9836    0.9160    0.9486       131

    accuracy                         0.9874      1034
   macro avg     0.9858    0.9569    0.9707      1034
weighted avg     0.9874    0.9874    0.9872      1034
```

The model reaches a recall of 99.78% on ham and a precision of 98.36% on spam. The weaker figure is the recall on spam (91.60%), because some spam messages are written to look like ordinary chat.

6.3.3 ROC Curves

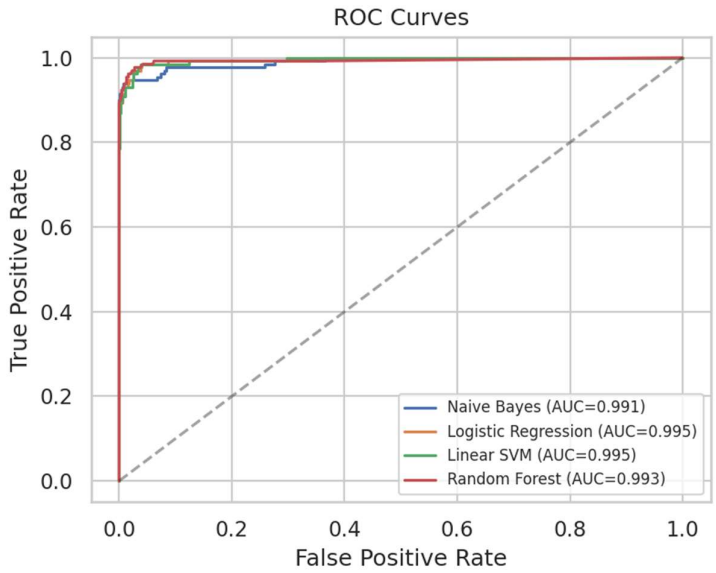


Figure 6.6 ROC curves of all four models

All curves rise steeply towards the top-left corner, and the area under each curve is above 0.99. This shows that, with a suitable threshold, each model can separate spam from ham very well.

6.3.4 Cross-Validation

To check that the result does not depend on one lucky split, five-fold stratified cross-validation was performed on the full dataset. The accuracy of the five folds was 98.26%, 98.65%, 98.65%, 98.74%, 98.64%, with a mean of 98.59%. The F1-score of the spam class had a mean of 94.22%. The small variation between folds shows that the model is stable.

6.4 Most Influential Words

Naive Bayes allows us to see which features the model considers most indicative of each class, by comparing the log-probabilities of every word under the two classes.

Table 6.3 Words with the strongest association to each class

Most indicative of SPAM	Most indicative of HAM
claim	ltgt

Most indicative of SPAM	Most indicative of HAM
prize	ill
won	lor
longnumber	da
18	later
500	lol
guaranteed	thats
1000	doing
150	ask
urltoken	really
tone	say
awarded	didnt

The spam words (claim, prize, won, guaranteed, awarded, ringtone, landline) match what is known about SMS scams: prizes, premium services and urgency. The special tokens 'longnumber' and 'urltoken', created in the cleaning step, also appear among the strongest spam indicators, which confirms that replacing numbers and links with tokens was a useful design choice. The ham words are informal conversational words ('lor', 'da', 'lol', 'ill', 'thats'), and 'ltgt' is a leftover from the encoding of the characters '<' and '>' in personal messages.

6.5 Error Analysis

The model made 2 false-positive and 11 false-negative errors on the test set. Looking at them explains the limitations of the approach.

False negatives (spam missed). Several missed messages are written in a friendly, personal tone with few typical spam words, for example a message that starts 'Hi babe its Jordan, how r u? Im home from abroad and lonely...' and then advertises a chat service, or a message that offers 'premium Pink Panther game ... all 4 FREE!' using unusual wording. Another uses capital letters and abbreviations about a mobile bill. Such messages hide their commercial purpose inside conversational language.

False positives (ham blocked). The wrongly blocked genuine messages were very short ('K k:) sms chat with me.' and 'Have you laid your airtel line to rest?'). They contain words such as 'sms', 'chat' and 'line', which commonly appear in spam about phone services. With so few words, the model has little other evidence.

These errors suggest that adding more training data, extra features such as message length or the number of capital letters, and context-aware models could improve the results further.

6.6 Final Prediction Output

The saved model was used to classify four new messages that were not part of the dataset. The output is shown below. The two spam-like messages (a free-phone offer with a link, and an urgent cash prize notice with a phone number) were correctly labelled SPAM, and the two normal messages (a lunch plan and a request for a report) were correctly labelled HAM.

```
FINAL PREDICTIONS
------------------------------------------------------------
[SPAM           ] Congratulations! You have won a free iPhone. Click http://win-
prize.com now to claim
[HAM (not spam) ] Hey, are we still meeting for lunch at 1 pm tomorrow?
[SPAM           ] URGENT! Your account is suspended. Call 09061701461 to claim
your 1000 pound cash prize
[HAM (not spam) ] Please send me the internship report draft by evening
```

6.7 Discussion

The results show that careful pre-processing and TF-IDF features matter more than model complexity for this problem. Even the simplest model reaches almost 99% accuracy. Naive Bayes was the best choice here for four reasons: it had the highest accuracy and F1-score, it needs very little computation, its decisions can be explained through word probabilities, and it can be retrained quickly as new spam patterns appear.

It should be noted that the results apply to this dataset, which consists of English SMS messages collected around 2011. Spam changes over time, and a filter used in practice would need to be retrained regularly and tested on current messages before deployment.

CHAPTER 7: CONCLUSION

This project developed a machine learning system that classifies text messages as spam or ham. The complete workflow was carried out: understanding and cleaning the dataset, removing duplicates, pre-processing the text, extracting TF-IDF features with unigrams and bigrams, training four different classifiers, evaluating them with several metrics, analysing the errors, and saving the best model for predicting new messages.

Multinomial Naive Bayes gave the best result, with an accuracy of 98.74%, a precision of 98.36%, a recall of 91.60% and an F1-score of 94.86%. Of 903 genuine messages in the test set, only 2 were wrongly blocked. Five-fold cross-validation gave a mean accuracy of 98.59%, showing that the performance is consistent.

The main lessons from the project are the following. First, the quality of data preparation has a larger effect than the choice of algorithm. Second, accuracy alone is not enough on imbalanced data, and precision, recall and F1-score must also be examined. Third, looking at the errors of a model gives insight that no single number can give. Fourth, a simple and explainable model can be as good as a more complex one when the features are well designed.

The project also helped me to gain practical skills in Python, pandas, scikit-learn, NLP pre-processing and model evaluation, and to compare it with my first internship project, Customer Churn Prediction, which worked on structured tabular data. Working with text showed me how different kinds of data need different pre-processing, while the overall machine learning workflow remains the same.

CHAPTER 8: FUTURE SCOPE

The system can be improved and extended in several directions:

- **Deep learning models.** Use word embeddings (Word2Vec, GloVe) or transformer models such as BERT to understand the context of the message and catch spam that is disguised as friendly conversation, which is where the current model makes most of its errors.
- **Additional features.** Add features such as message length, number of capital letters, count of digits, presence of currency symbols and exclamation marks, and combine them with the TF-IDF features.
- **Larger, multilingual data.** Train on larger and more recent datasets, including email and messages in Hindi and Hinglish, because spam in India is often written in mixed languages.
- **Web application and API.** Wrap the model in a web application using Streamlit or Flask, or expose it as a REST API, so that users can paste a message and see the result instantly.
- **Real-time deployment.** Build a browser extension or mobile app that scans incoming messages and warns the user before they click a link.
- **Continuous learning.** Retrain the model regularly, using messages that users report as spam, so that it adapts to new scams.
- **Threshold tuning and confidence.** Tune the decision threshold to reduce false positives further, and show a confidence score to the user instead of a hard label.
- **Explainability.** Use LIME or SHAP to show which words led to a decision, so that users and administrators can trust the filter.
- **Link analysis.** Extend the system to detect phishing links by analysing the URLs found in messages.

REFERENCES

[1] Almeida, T. A., Gomez Hidalgo, J. M., and Yamakami, A. (2011). Contributions to the study of SMS spam filtering: new collection and results. Proceedings of the 11th ACM Symposium on Document Engineering (DocEng '11), pp. 259-262.

[2] Sahami, M., Dumais, S., Heckerman, D., and Horvitz, E. (1998). A Bayesian approach to filtering junk e-mail. AAAI Workshop on Learning for Text Categorization.

[3] Androutsopoulos, I., Koutsias, J., Chandrinos, K. V., Paliouras, G., and Spyropoulos, C. D. (2000). An evaluation of Naive Bayesian anti-spam filtering. Workshop on Machine Learning in the New Information Age, 11th European Conference on Machine Learning.

[4] Graham, P. (2002). A Plan for Spam. http://www.paulgraham.com/spam.html

[5] Drucker, H., Wu, D., and Vapnik, V. N. (1999). Support vector machines for spam categorization. IEEE Transactions on Neural Networks, 10(5), 1048-1054.

[6] Cortes, C., and Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297.

[7] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[8] Salton, G., and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing and Management, 24(5), 513-523.

[9] Manning, C. D., Raghavan, P., and Schutze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

[10] Cormack, G. V. (2008). Email spam filtering: a systematic review. Foundations and Trends in Information Retrieval, 1(4), 335-455.

[11] Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.

[12] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019.

[13] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[14] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.

[15] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science and Engineering, 9(3), 90-95.

[16] UCI Machine Learning Repository. SMS Spam Collection Dataset. https://archive.ics.uci.edu/dataset/228/sms+spam+collection

APPENDIX: COMPLETE SOURCE CODE

The complete script (spam_detection.py) used in this project is listed below. The same code, in step-by-step form with outputs, is available in the notebook Spam_Mail_Prediction.ipynb.

```
"""
Spam Mail Prediction using Machine Learning
Author : Shashank Deshmukh
Internship : Naviotech Solutions Pvt Ltd (AI & ML)

Pipeline: load data -> EDA -> clean text -> TF-IDF -> train 4 models
          -> evaluate -> save best model -> predict on new messages
"""

import json
import re
import string
import warnings

import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, f1_score, precision_score,
                             recall_score, roc_auc_score, roc_curve)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42

# ---------------------------------------------------------------- 1. Load
df = pd.read_csv("sms.tsv", sep="\t", header=None, names=["label", "message"])
print("Dataset shape:", df.shape)
print(df.head(), "\n")
print(df["label"].value_counts(), "\n")
print("Missing values:\n", df.isnull().sum(), "\n")
print("Duplicates:", df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
print("Shape after removing duplicates:", df.shape, "\n")

df["target"] = df["label"].map({"ham": 0, "spam": 1})
df["length"] = df["message"].str.len()
df["words"] = df["message"].str.split().str.len()

# ---------------------------------------------------------------- 2. EDA plots
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
counts = df["label"].value_counts()
ax[0].bar(counts.index, counts.values, color=["#2e86ab", "#e4572e"])
ax[0].set_title("Class Distribution")
ax[0].set_ylabel("Number of messages")
for i, v in enumerate(counts.values):
    ax[0].text(i, v + 30, str(v), ha="center")
ax[1].pie(counts.values, labels=counts.index, autopct="%1.1f%%",
```

```
          colors=["#2e86ab", "#e4572e"], startangle=90)
ax[1].set_title("Ham vs Spam Share")
plt.tight_layout()
plt.savefig("fig_class_distribution.png", dpi=150)
plt.close()

plt.figure(figsize=(8, 4))
sns.histplot(data=df, x="length", hue="label", bins=50,
             palette=["#2e86ab", "#e4572e"], element="step")
plt.title("Message Length Distribution")
plt.xlabel("Characters per message")
plt.tight_layout()
plt.savefig("fig length distribution.png", dpi=150)
plt.close()



# ---------------------------------------------------------------- 3. Clean
def clean_text(text: str) -> str:
    text = text.lower()
    text = re.sub(r"http\S+|www\.\S+", " urltoken ", text)
    text = re.sub(r"\d{5,}", " longnumber ", text)
    text = text.translate(str.maketrans("", "", string.punctuation))
    tokens = [w for w in text.split() if w not in ENGLISH_STOP_WORDS]
    return " ".join(tokens)



df["clean"] = df["message"].apply(clean_text)
print("Cleaning example:")
print(" before:", df["message"][2])
print(" after :", df["clean"][2], "\n")

# top words per class
def top_words(series, n=12):
    return pd.Series(" ".join(series).split()).value_counts().head(n)

tw_ham = top_words(df[df.target == 0]["clean"])
tw_spam = top_words(df[df.target == 1]["clean"])
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
tw_ham[::-1].plot.barh(ax=ax[0], color="#2e86ab")
ax[0].set_title("Top words in HAM")
tw_spam[::-1].plot.barh(ax=ax[1], color="#e4572e")
ax[1].set_title("Top words in SPAM")
plt.tight_layout()
plt.savefig("fig_top_words.png", dpi=150)
plt.close()

# ---------------------------------------------------------------- 4. Split
X_train, X_test, y_train, y_test = train_test_split(
    df["clean"], df["target"], test size=0.2,
    stratify=df["target"], random_state=RANDOM_STATE)
print("Train size:", len(X_train), "| Test size:", len(X_test), "\n")

# ---------------------------------------------------------------- 5. Models
def make_pipe(clf):
    return Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2,
sublinear_tf=True)),
        ("clf", clf),
    ])

models = {
    "Naive Bayes": MultinomialNB(alpha=0.1),
    "Logistic Regression": LogisticRegression(max_iter=1000, C=10,
                                              class_weight="balanced"),
    "Linear SVM": LinearSVC(C=1, class_weight="balanced",
                            random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(n_estimators=200, n_jobs=-1,
                                            random_state=RANDOM_STATE),
```

```
}

results, fitted, preds, scores = [], {}, {}, {}
for name, clf in models.items():
    pipe = make_pipe(clf)
    pipe.fit(X_train, y_train)
    y pred = pipe.predict(X test)
    if hasattr(pipe, "predict_proba"):
        s = pipe.predict_proba(X_test)[:, 1]
    else:
        s = pipe.decision_function(X_test)
    fitted[name], preds[name], scores[name] = pipe, y_pred, s
    row = {
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1": f1_score(y_test, y_pred),
        "ROC_AUC": roc_auc_score(y_test, s),
    }
    results.append(row)
    print(f"{name:20s} acc={row['Accuracy']:.4f} prec={row['Precision']:.4f} "
          f"rec={row['Recall']:.4f} f1={row['F1']:.4f} auc={row['ROC_AUC']:.4f}")

res_df = pd.DataFrame(results).sort_values("F1",
ascending=False).reset_index(drop=True)
print("\n", res_df.round(4).to_string(index=False), "\n")
res_df.to_csv("model_comparison.csv", index=False)

# ---------------------------------------------------------------- 6. Plots
plt.figure(figsize=(8, 4.5))
x = np.arange(len(res_df))
w = 0.2
for i, m in enumerate(["Accuracy", "Precision", "Recall", "F1"]):
    plt.bar(x + i * w, res_df[m], w, label=m)
plt.xticks(x + 1.5 * w, res df["Model"], rotation=12)
plt.ylim(0.7, 1.02)
plt.title("Model Comparison")
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("fig_model_comparison.png", dpi=150)
plt.close()

best_name = res_df.loc[0, "Model"]
best = fitted[best_name]
print("Best model:", best_name, "\n")

cm = confusion_matrix(y_test, preds[best_name])
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Ham", "Spam"], yticklabels=["Ham", "Spam"])
plt.title(f"Confusion Matrix - {best name}")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.savefig("fig_confusion_matrix.png", dpi=150)
plt.close()

plt.figure(figsize=(5.5, 4.5))
for name in fitted:
    fpr, tpr, _ = roc_curve(y_test, scores[name])
    plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test,
scores[name]):.3f})")
plt.plot([0, 1], [0, 1], "k--", alpha=0.4)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves")
plt.legend(fontsize=8)
plt.tight_layout()
```

```
plt.savefig("fig_roc_curves.png", dpi=150)
plt.close()

report = classification_report(y_test, preds[best_name],
                               target_names=["Ham", "Spam"], digits=4)
print(report)

# ---------------------------------------------------------------------- 7. Save
joblib.dump(best, "spam_model.joblib")
tn, fp, fn, tp = cm.ravel()
summary = {
    "total_messages": int(len(df)),
    "ham": int((df.target == 0).sum()),
    "spam": int((df.target == 1).sum()),
    "train": int(len(X_train)),
    "test": int(len(X_test)),
    "best_model": best_name,
    "confusion": {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)},
    "report": report,
    "results": res_df.round(4).to_dict(orient="records"),
}
with open("metrics.json", "w") as f:
    json.dump(summary, f, indent=2)



# ---------------------------------------------------------------------- 8. Predict
def predict_message(msg: str):
    model = joblib.load("spam_model.joblib")
    label = model.predict([clean_text(msg)])[0]
    return "SPAM" if label == 1 else "HAM (not spam)"



samples = [
    "Congratulations! You have won a free iPhone. Click http://win-prize.com now
to claim",
    "Hey, are we still meeting for lunch at 1 pm tomorrow?",
    "URGENT! Your account is suspended. Call 09061701461 to claim your 1000 pound
cash prize",
    "Please send me the internship report draft by evening",
]
print("FINAL PREDICTIONS")
print("-" * 60)
for s in samples:
    print(f"[{predict_message(s):15s}] {s}")
```